EXP NO: 9 DATE:	CI/CD PIPELINES IN AZURE

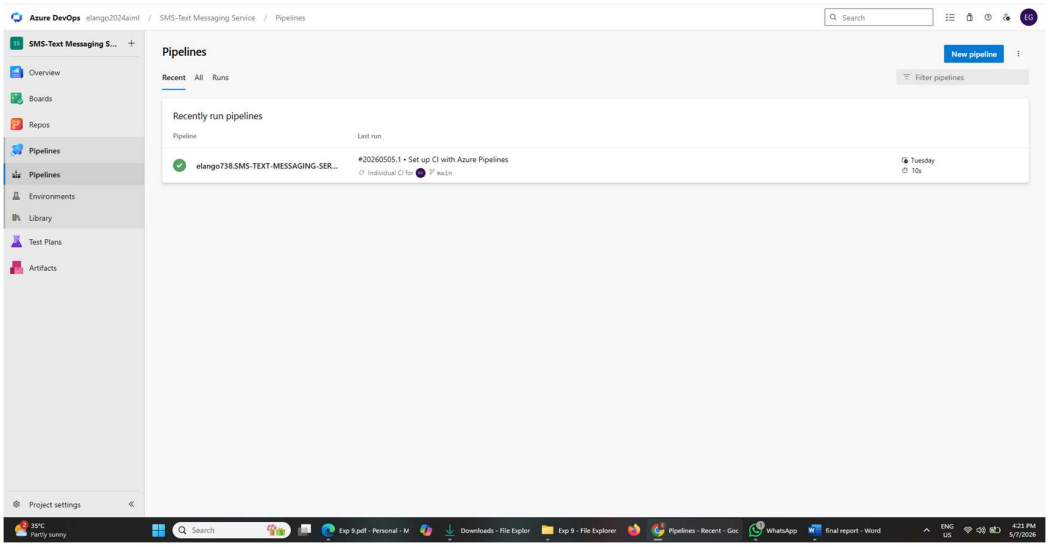
AIM: To implement a Continuous Integration and Continuous Deployment (CI/CD) pipeline in Azure DevOps for automating the build, testing, and deployment process of the Student Management System, ensuring faster delivery and improved software quality.

PROCEDURE:

Steps to Create and implement pipelines in Azure:

1. **Sign in to Azure DevOps and Navigate to Your Project** Log in to dev.azure.com, select your organization, and open the project where your Student Management System code resides.
2. **Connect a Code Repository (Azure Repos or GitHub)** Ensure your application code is stored in a Git-based repository such as Azure Repos or GitHub. This will be the source for triggering builds and deployments in your pipeline.
3. **Create a New Pipeline** Go to the Pipelines section on the left panel and click "Create Pipeline". Choose your source (e.g., Azure Repos Git or GitHub), and then select the repository containing your project code.
4. **Choose the Pipeline Configuration** You can select either the YAML-based pipeline (recommended for version control and automation) or the Classic Editor for a GUI-based setup. If using YAML, Azure DevOps will suggest a template or allow you to define your own.
5. **Define Build Stage (CI - Continuous Integration) from YAML file**
6. Install dependencies (e.g., npm install, dotnet restore)
7. Build the application (dotnet build, npm run build)
8. Run unit tests (dotnet test, npm test)
9. Publish build artifacts to be used in the release stage

10. **Save and Run the Pipeline for the First Time** Save the YAML or build definition and click "Run". Azure will fetch the latest code and execute the defined build and test stages.
11. **Configure Continuous Deployment (CD)** Navigate to the Releases tab under Pipelines and click "New Release Pipeline". Add an Artifact (from the build stage) and create a new Stage (e.g., Development, Production).
12. Configure the CD stage with deployment tasks such as deploying to Azure App Service, running database migrations or scripts, and restarting services using the Azure App Service Deploy task linked to your subscription and app details.
13. **Set Triggers and Approvals** Enable continuous deployment trigger so the release pipeline runs automatically after a successful build. For production environments, configure pre-deployment approvals to ensure manual verification before release.
14. **Monitor Pipelines and Manage Logs** View all pipeline runs under the Runs section. Check logs for build/test/deploy stages to debug any errors. You can also integrate email alerts or Microsoft Teams notifications for build failures.
15. **Review and Maintain Pipelines** Regularly update your pipeline tasks or YAML configurations as your application grows. Ensure pipeline runs are clean and artifacts are stored securely. Integrate quality gates and code coverage policies to maintain code quality.



RESULT: Thus ,the pipelines for the given project " Online Marketplace Platform " has been executed successfully.